
MUSHROOMCLASSIFICATIONUSINGNAIVEBAYES CLASSIFIERBASEDLEARNINGMODEL

Ishaan Singla
(24103099)

1.) Introduction

Of more than millions of mushroom species growing all over the world , one type is edible while the other is poisonous. It is not easy to distinguish them from each other , hence it needs expertise to do so due to which the classification of mushrooms into edible and poisonous is important . Machine learning models can be an alternate method for classifying the mushrooms. In our case , we will be using Naive Bayes Classifier although decision trees , ada boost etc. can also be used.

2.) Dataset

The dataset mushroom.csv has 22 input columns and 1 output column named 'Class' which can take only values - 'e' or 'p' where these values denote edible and poisonous respectively. All the 22 input values are categorical and we have got 8124 data points in total in which we have not got any NULL values in any of the columns. The dataset is balanced having almost 51.79% of total mushroom entries as edible. We apply one hot encoding to change the categorical values from strings to numerics.

3.) Methods and Experiments

We employed a Naive Bayes Classifier Model to predict whether the mushroom is edible or poisonous in the mushroom.csv dataset. The implementation of the Gaussian variant of the Naive Bayes Classifier is critical owing to the continuous nature of the attribute values of the dataset but in our case , all the attributes have categorical, and hence discrete values . So , categorical naive bayes would give better results in our case. We will be using only the numpy library to implement our algorithm and then compare the results with the scikit learn naive bayes classifier .

3.1) Naive Bayes

The Naive Bayes classifier assumes that the presence of a particular feature in a class has no relation with the presence of any other feature, that is all the attributes are independent of each other. This assumption is called "naive" because it is too simplistic and may not be true in real-world situations. However, despite its simplicity, Naive Bayes often performs surprisingly well in practice and is computationally efficient also, particularly in text classification and spam filtering tasks.

The algorithm uses Bayes' theorem to calculate the probability of each class given a set of features. It then selects the class with the highest probability as the predicted class for the input data. Naive Bayes is particularly efficient for large datasets and high-dimensional feature spaces. Bayes Theorem provides means for calculating posterior probability from prior probabilities.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

The diagram shows the formula for Bayes' Theorem. The numerator consists of two terms: $P(B|A)$ and $P(A)$. $P(B|A)$ is labeled 'Likelihood' with a line pointing to it. $P(A)$ is labeled 'Prior' with a line pointing to it. The denominator is $P(B)$, which is labeled 'Evidence' with a line pointing to it. The entire fraction is labeled 'Posterior' with a line pointing to it.

We then applied the above formula where A was our output variable where A1 and A2 are the two possible outcomes . and B1, B2 ,..., B22 are our 22 input variables . We used the 'naive' assumption that the attributes are independent of each other and used the conditional

independence property to write it as : $P(A_j | \prod_{i=1}^k B_i) = P(A_j) \cdot \prod_{i=1}^k P(B_i | A_j)$ where i ranges from 1 to 22 and j from 1 to 2 .

We took the maximum value across each class for a given input of variables and then accordingly denoted the output of given input value as edible or poisonous .

If we find strong correlation among the variables, it will go against our 'naive' assumption that attributes are independent from each other. Note , in the above formula, every attribute has an equal importance. So, if our correlated features happen to be a good predictor , the model will actually be benefited from it and if our correlated features happen to be a bad predictor , the model will be worse off . Attributes which have very less correlation with the output variable can be removed to get better results.

4.) Results

The comparative results between our model and the implementation of the scikit learn categorical naive bayes model can be found below .

Our Model :

	precision	recall	f1-score
edible	0.52	1.00	0.68
poisonous	0.00	0.00	0.00

Accuracy - 53.72 %

Scikit Learn Categorical Naive Bayes Classifier :

	precision	recall	f1-score
edible	0.90	0.98	0.94
poisonous	0.98	0.89	0.93

Accuracy - 94%